# Lab 8. Basic of Machine Learning with scikit-learn

In this mini-project, we will explore the basic workflow of machine learning (ML) using the `scikit-learn` library in Python.

We will work with a simple dataset and walk step by step through a classification problem following this pipeline:

- Load and prepare the data
- Perform preliminary analysis
- Split into training and test sets
- Standardize the data
- Train the k-Nearest Neighbors Model
- Make predictions
- Evaluate model performance

## Iris dataset

We will use the classic Iris dataset, which contains 150 flower observations with four numerical features: sepal length (cm), sepal width (cm), petal length (cm), petal width (cm). The target variable is the species of the iris flower ( `setosa` , `versicolor` , or `virginica` ).

We load the data using the following code:

In [2]:
```
from sklearn.datasets import load_iris
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

df = load_iris(as_frame=True).frame
df.head()
```

Out[2]:

| | sepal length (cm) | sepal width (cm) | petal length (cm) | petal width (cm) | target |
|---|---|---|---|---|---|
| **0** | 5.1 | 3.5 | 1.4 | 0.2 | 0 |
| **1** | 4.9 | 3.0 | 1.4 | 0.2 | 0 |
| **2** | 4.7 | 3.2 | 1.3 | 0.2 | 0 |
| **3** | 4.6 | 3.1 | 1.5 | 0.2 | 0 |
| **4** | 5.0 | 3.6 | 1.4 | 0.2 | 0 |

## Task 1. (Check the data)

The very first step is checking the data.

---

**Instructions:**

Print the following information:

- The shape of the dataset (rows × columns).
- Summary statistics using `describe()`.
- Count the number of observations in each target class (`setosa`, `versicolor`, or `virginica`).
- Count the number of missing values.

In [3]:
```
print(f"Shape of our dataframe: {df.shape}.")
print(f"\nSummary statistics:\n{df.describe()}.")
print(f"\nNumber of each target (type of flower):\n{df.groupby('target').size()}")
print(f"\nWe can see there are no NA's:\n{df.isna().sum()}.")
```

```
Shape of our dataframe: (150, 5).

Summary statistics:
       sepal length (cm)  sepal width (cm)  petal length (cm)  \
count         150.000000        150.000000         150.000000
mean            5.843333          3.057333           3.758000
std             0.828066          0.435866           1.765298
min             4.300000          2.000000           1.000000
25%             5.100000          2.800000           1.600000
50%             5.800000          3.000000           4.350000
75%             6.400000          3.300000           5.100000
max             7.900000          4.400000           6.900000

       petal width (cm)      target
count        150.000000  150.000000
mean           1.199333    1.000000
std            0.762238    0.819232
min            0.100000    0.000000
25%            0.300000    0.000000
50%            1.300000    1.000000
75%            1.800000    2.000000
max            2.500000    2.000000  .

Number of each target (type of flower):
target
0    50
1    50
2    50
dtype: int64

We can see there are no NA's:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target               0
dtype: int64.
```

## Task 2. (Preliminary Analysis)

The next step is to perform a preliminary analysis and visualize the data. This helps us better understand feature distributions, relationships, and possible correlations.

---

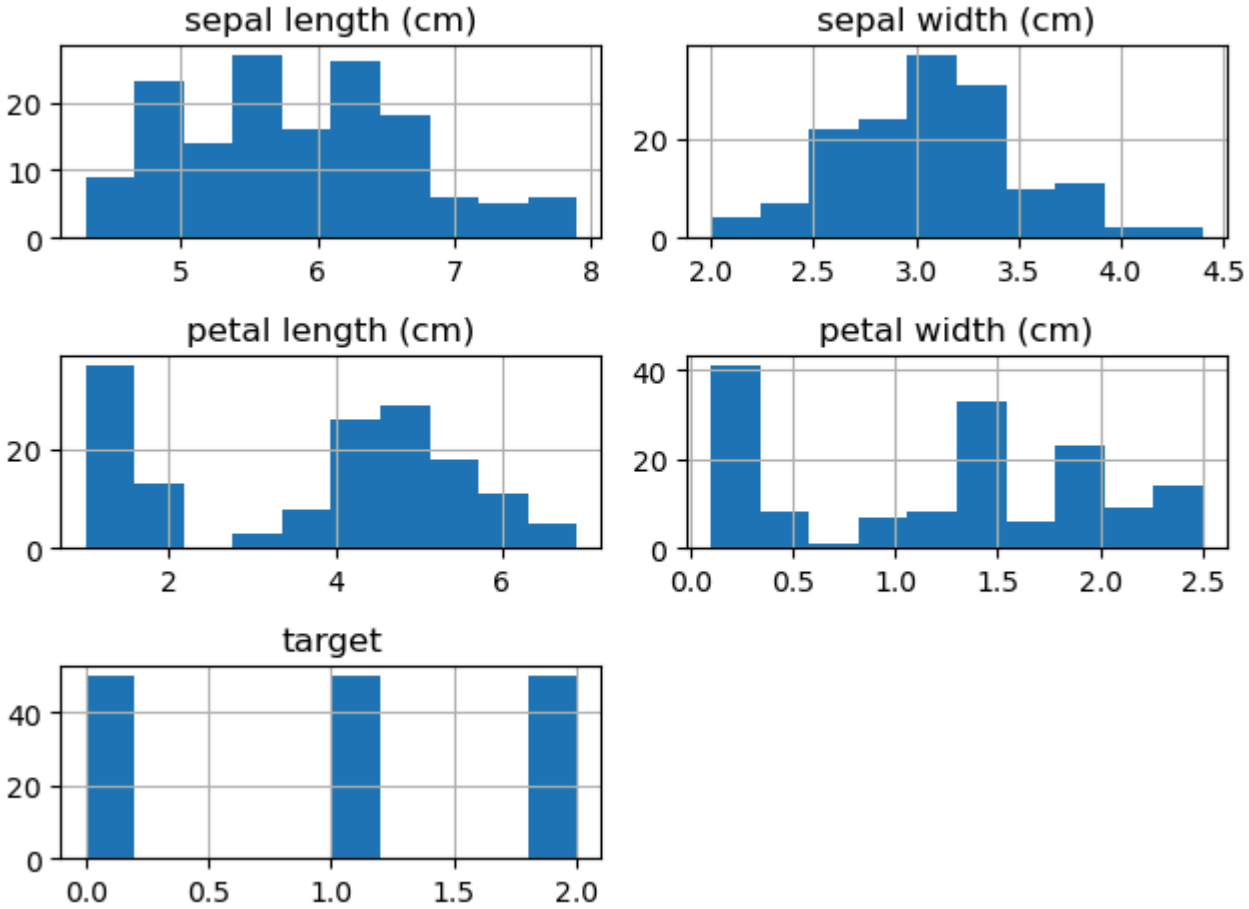
**Instructions:**:

Create the following visualizations:

- Histograms of all numerical features (use `df.hist`)
- Pairplot of feature pairs, colored by target class (use `sns.pairplot`)
- Correlation heatmap with annotated values (compute the correlation matrix and use `sns.heatmap`)

Histograms:

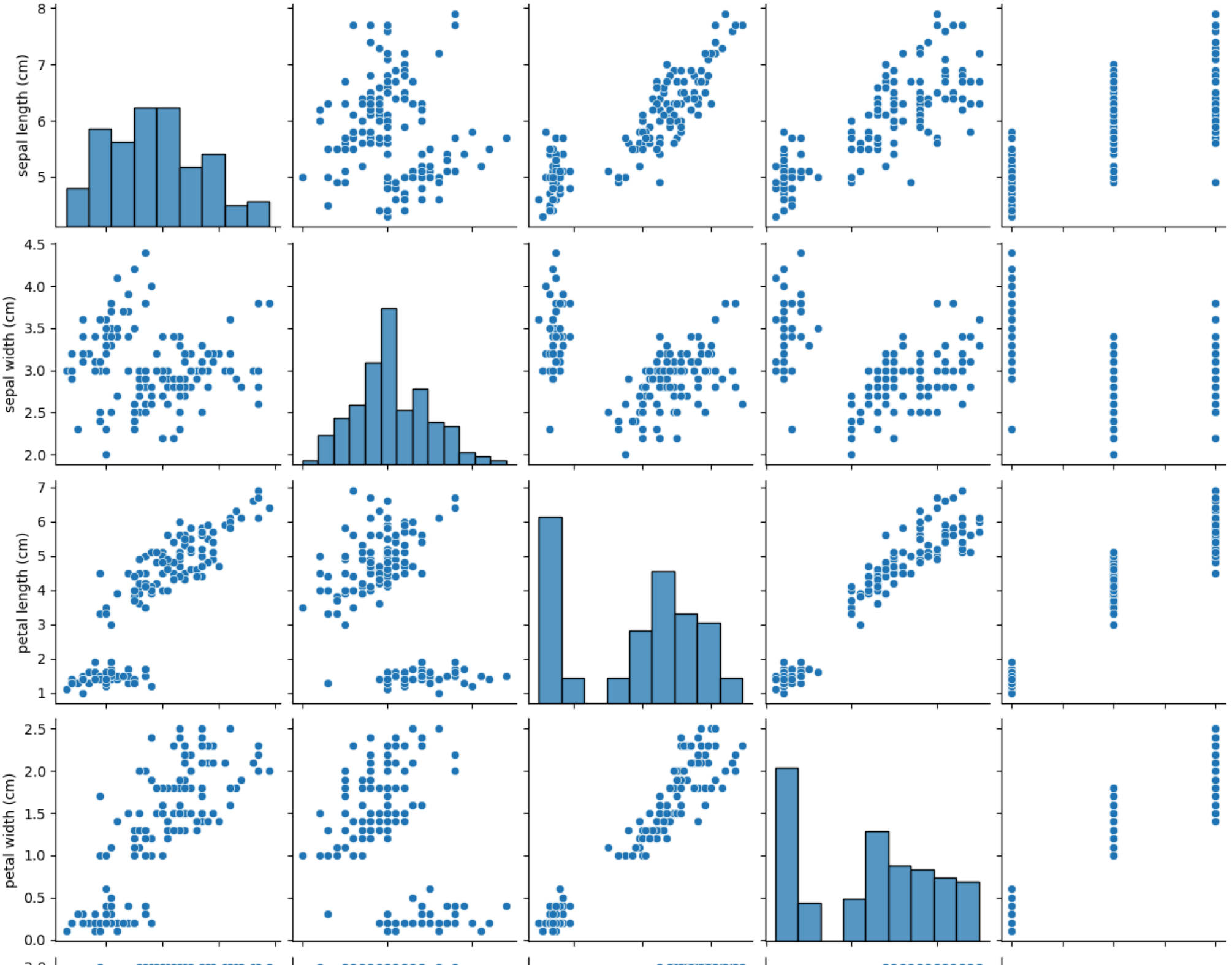
In [4]:
```
df.hist()
plt.tight_layout()
plt.show()
```

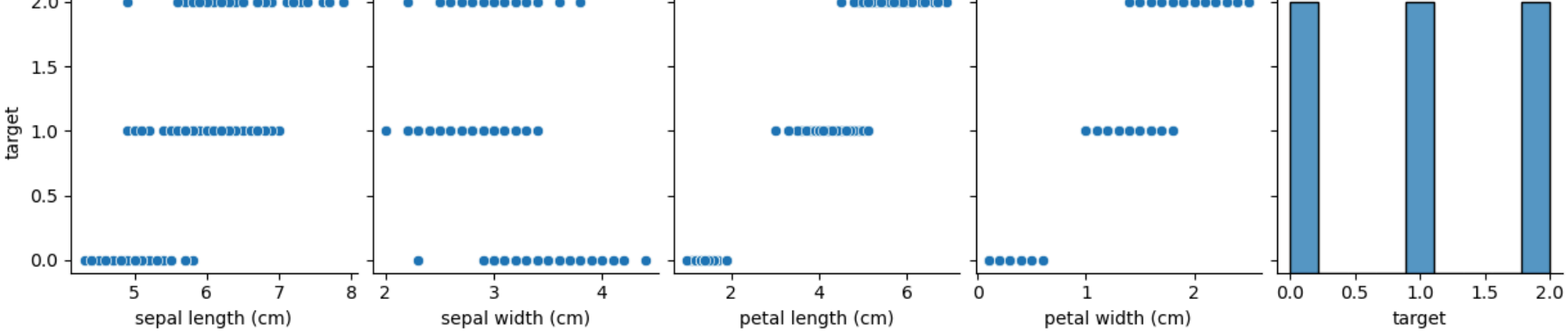



Pairplots (there is a warning, but it doesn't affect our aim - clean pairplots):

In [5]:
```
sns.pairplot(df)
plt.show()
```

```
/opt/conda/envs/anaconda-panel-2023.05-py310/lib/python3.11/site-packages/seaborn/axisgrid.py:118: UserWarning: The figure layout has change
d to tight
  self._figure.tight_layout(*args, **kwargs)
```


sepal length (cm)
sepal width (cm)
petal length (cm)
petal width (cm)

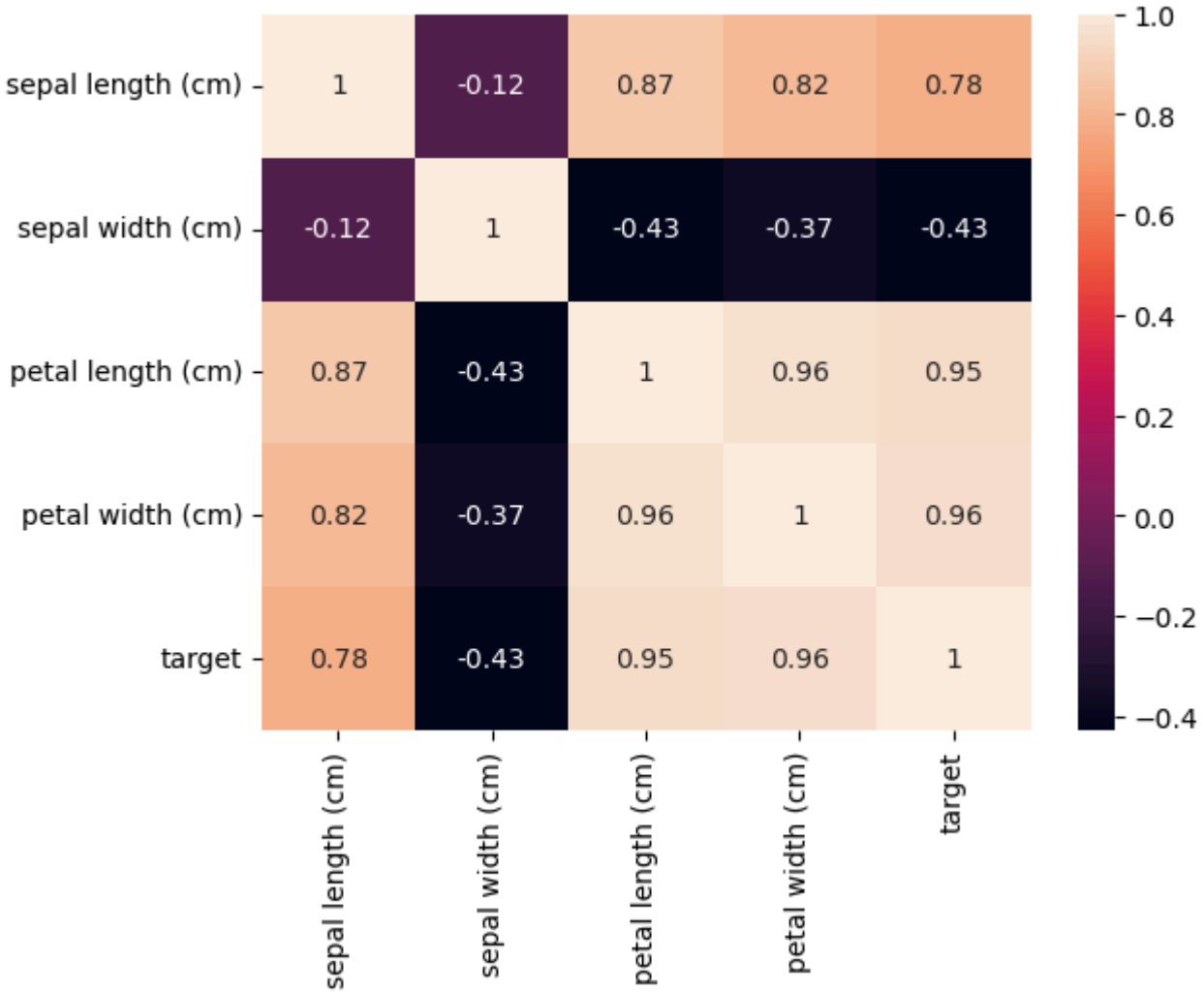



Correlation heatmap:

In [6]:
```
corr_matrix=df.corr()
print(f"Correlation matrix:\n{corr_matrix}")

print("\n\nHeatmap:\n")
sns.heatmap(corr_matrix, annot=True)
```

```
Correlation matrix:
                   sepal length (cm)  sepal width (cm)  petal length (cm)  \
sepal length (cm)           1.000000         -0.117570           0.871754
sepal width (cm)           -0.117570          1.000000          -0.428440
petal length (cm)           0.871754         -0.428440           1.000000
petal width (cm)            0.817941         -0.366126           0.962865
target                      0.782561         -0.426658           0.949035

                   petal width (cm)    target
sepal length (cm)          0.817941  0.782561
sepal width (cm)          -0.366126 -0.426658
petal length (cm)          0.962865  0.949035
petal width (cm)           1.000000  0.956547
target                     0.956547  1.000000


Heatmap:
```

Out[6]: <Axes: >




## Task 3. (Split the data into training and test sets)

Now, let us split the dataset into two parts: a training set (used to fit the model) and a test set (used to evaluate the model). This is a crucial step to ensure that our model is tested on unseen data.

---

**Instructions:**

- Separate the dataset into features ( `X` ) and target ( `y` ).
- Use `train_test_split` from `sklearn.model_selection` to divide the data into training (80%) and test (20%) sets.

- Make sure to use the parameter `stratify=y` to preserve class proportions.
- Set `random_state=42` for reproducibility.
- Print the shapes of the training and test sets.

In [7]:
```
from sklearn.model_selection import train_test_split
X=df.iloc[:,:-1]
y=df.iloc[:,-1]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
) #stratify=y ensures us that both our sets will have similiar ratio of each target (cool!)

print(f"X_train shape: {X_train.shape}.")
print(f"X_test shape: {X_test.shape}.")
print(f"y_train shape: {y_train.shape}.")
print(f"y_test shape: {y_test.shape}.")
```

```
X_train shape: (120, 4).
X_test shape: (30, 4).
y_train shape: (120,).
y_test shape: (30,).
```

## Task 4. (Standardize the data)

The next step is to standardize the data, i.e.,

$$\frac{X-\mu}{\sigma}$$

This step is especially important for algorithms that are sensitive to feature scales, such as kNN, Logistic Regression, or SVM.

---

**Remark:** Fit the scaler only on the training set, then transform both train and test. This prevents data leakage (using information from the test set during training).

---

**Instructions:**

- Use `StandardScaler` from `sklearn.preprocessing`.
- Fit the scaler on the training set and transform it (use `fit_transform()`).

- Transform the test set using the same scaler (use `.transform()`).
- Check the mean and standard deviation for scaled train dataset.

---

**Hints:**

`StandardScaler` is a class in `sklearn.preprocessing`. First, we should create a new object (variable) from this class `scaler = StandardScaler()`. Then we can use the methods `fit`, `transform` and `fit_transform`:

- `scaler.fit(X)` - learns the mean and standard deviation of features in `X`.
- `scaler.transform(X)` - applies scaling using the stored values (from `fit`).
- `scaler.fit_transform(X)` - runs `fit` and then `transform` in one step (useful for training data).

In [8]:
```
from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
X_train_scaled=scaler.fit_transform(X_train)
X_test_scaled=scaler.transform(X_test)

print(f"Train dataset was normalised: mean: {round(np.mean(X_train_scaled),3)}, standard deviation: {round(np.std(X_train_scaled),3)}")
```

```
Train dataset was normalised: mean: 0.0, standard deviation: 1.0
```

## The k-Nearest Neighbors (kNN) Model

The k-Nearest Neighbors algorithm is one of the simplest machine learning models. It does not build an explicit equation like linear regression or logistic regression. Instead, it makes predictions by looking at the closest data points in the training set.

## Mathematical Formulation

We have a dataset:

$$\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$$

where $x_i \in \mathbb{R}^d$ is a vector of features, and $y_i \in \{1, 2, \ldots, C\}$ is the class label.

---

### Distance function

To measure similarity between two points, we use a **distance metric**. Common choices:

- **Euclidean distance ($L^2$ norm):**

$$d(x, x_i) = \sqrt{\sum_{j=1}^{d} (x_j - x_{ij})^2}$$

- **Manhattan distance ($L^1$ norm):**

$$d(x, x_i) = \sum_{j=1}^{d} |x_j - x_{ij}|$$

---

### Classification rule

1. For a new observation $x$, compute the distance $d(x, x_i)$ to every training point $x_i$.
2. Select the set of **k nearest neighbors**:

$$N_k(x) = \{(x_i, y_i) : x_i \text{ is among the k closest points to } x\}$$

3. Predict the **majority class** among those neighbors:

$$\hat{y}(x) = \arg \max_{c \in \{1,\ldots,C\}} \sum_{(x_i, y_i) \in N_k(x)} \mathbf{1}(y_i = c)$$

where $\mathbf{1}(\cdot)$ is the indicator function.

---

### Regression with kNN

If the target variable is continuous $y \in \mathbb{R}$, prediction is the **average** (or weighted average) of neighbors:

$$\hat{y}(x) = \frac{1}{k} \sum_{(x_i, y_i) \in N_k(x)} y_i$$

---

### Effect of parameter k

- Small $k$: low bias, high variance (very flexible, but sensitive to noise).
- Large $k$: high bias, low variance (smoother decision boundary, but less adaptive).

## Task 5. (Implement a kNN from scratch)

In this task, you will implement the k-Nearest Neighbors (kNN) algorithm manually. This will help you better understand how the method works behind the scenes.

---

**Instructions:**

- Write a function `kNN` such that:
  - Takes as input:
    - training features `X_train_scaled` and labels `y_train`,
    - test features `X_test_scaled`,
    - parameter `k` (number of neighbors).
  - For each test sample:
    - Compute the distance between this test sample and all training samples (use Euclidean distance):

$$d(x, x_i) = \sqrt{\sum_{j=1}^{d} (x_j - x_{ij})^2}$$

    - Find the $k$ nearest neighbors (the training points with the smallest distances).
    - Select the majority class among these neighbors. The prediction rule is:

$$\hat{y}(x) = \arg\max_{c \in \{1,\ldots,C\}} \sum_{i \in N_k(x)} \mathbf{1}(y_i = c)$$

      where $N_k(x)$ is the set of indices of the $k$ nearest neighbors of $x$, and $\mathbf{1}(\cdot)$ is the indicator function.
  - Returns predicted labels for all test samples.
- Finally, evaluate your function and compute the accuracy score using the formula (without explicit loops, use vectorized operations instead):

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

---

**Hints:**

- Convert the data into a numpy array using the `to_numpy` function.

- Use `np.linalg.norm` to compute distances between points.
- Use `np.argsort` to find indices of the $k$ smallest distances.
- Use `np.bincount` with `np.argmax` to find the majority class.

In [20]:
```
y_train.to_numpy()
y_test.to_numpy()

def kNN(x_train, y_train, x_test, k):
    predictions=np.zeros(len(x_test))
    for i in range(len(x_test)):
        distances=[np.linalg.norm(x_test[i] - training_sam) for training_sam in x_train]
        order=np.argsort(distances)[:k] #indices returning k smallest distances
        #y_train.iloc[order] <- y values for those indices
        winner=np.argmax(np.bincount(y_train.iloc[order])) #the most represented class in "order"
        predictions[i]=winner
    return(predictions)

def accuracy(y_pred, y_test):
    n=len(y_test)
    counter=np.where(y_pred==y_test, 1, 0)
    acc=sum(counter)/n
    return(acc)

y_pred_my_own_knn=kNN(X_train_scaled, y_train, X_test_scaled, 5)
print(f"Our self-written kNN accuracy: {round(accuracy(y_pred_my_own_knn, y_test),6)}.")
```

```
Our self-written kNN accuracy: 0.933333.
```

## Task 6. (Train and fit a kNN using `scikit-learn`)

Now, let us train a k-Nearest Neighbors (kNN) classifier.

---

**Instructions:**

- Use `KNeighborsClassifier` from `sklearn.neighbors`.
- Fit the model on the standardized training set (`X_train_scaled`, `y_train`) with $k = 5$.
- Predict labels for the standardized test set (`X_test_scaled`).
- Compute the accuracy score using `accuracy_score` from `sklearn.metrics`.

**Hints:**

`KNeighborsClassifier` is a class in `sklearn.neighbors`. First, we should create a new object (variable) from this class `knn = KNeighborsClassifier(...)`. Then use its main methods:

- `knn.fit(X, y)` – fits the classifier to the training data (`X` = features, `y` = labels).
- `knn.predict(X)` – predicts class labels for new data `X` using the fitted model.

In [21]:
```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
knn=KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
y_pred=knn.predict(X_test_scaled)


print(f"scikit kNN accuracy: {round(accuracy_score(y_pred, y_test),6)}.")
```

```
scikit kNN accuracy: 0.933333.
```

## Alternatively...

Alternatively, we can use a `Pipeline`. A pipeline allows us to chain multiple steps (e.g., preprocessing + model) into a single object. This is very convenient, because it ensures that the same preprocessing (like scaling) is applied consistently during both training and prediction.

**Example:**

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

knn_model = Pipeline([
    ("scaler", StandardScaler()),          # standardize features
    ("knn", KNeighborsClassifier(n_neighbors=5))  # kNN classifier
])

knn_model.fit(X_train, y_train)     # fit scaler + model on training set
y_pred = knn_model.predict(X_test)  # apply scaler + model to test set
```

## Task 7. (Detailed evaluation)

After training the kNN classifier, we want to check how well it performs for each class.

---

**Mathematical definitions:**

For a given class $c$:

- True Positives (TP): correctly predicted as class $c$
- False Positives (FP): predicted as class $c$, but actually another class
- False Negatives (FN): actually class $c$, but predicted as another class

Then the evaluation metrics are:

$$\text{Precision}(c) = \frac{TP}{TP + FP}$$

$$\text{Recall}(c) = \frac{TP}{TP + FN}$$

$$\text{F1}(c) = 2 \cdot \frac{\text{Precision}(c) \cdot \text{Recall}(c)}{\text{Precision}(c) + \text{Recall}(c)}$$

---

**Instructions:**

- Compute $TP$, $FP$, $FN$, $\text{Precision}$, $\text{Recall}$, and $\text{F1}$ for a class $c = 0$ manually using the formulas above (without loops).
- Print a classification report showing: precision, recall, F1-score, and support (number of true samples in each class) (use `classification_report`).
- Plot a confusion matrix to visualize which classes are correctly classified and where the model makes mistakes (use `confusion_matrix` and `ConfusionMatrixDisplay`).

---

**Functions to use (from `sklearn.metrics`):**

- `classification_report(y_true, y_pred)` – prints Precision, Recall, F1-score, and Support.
- `confusion_matrix(y_true, y_pred)` – returns the confusion matrix as a 2D array.
- `ConfusionMatrixDisplay(confusion_matrix=cm)` – convenient way to plot the confusion matrix.

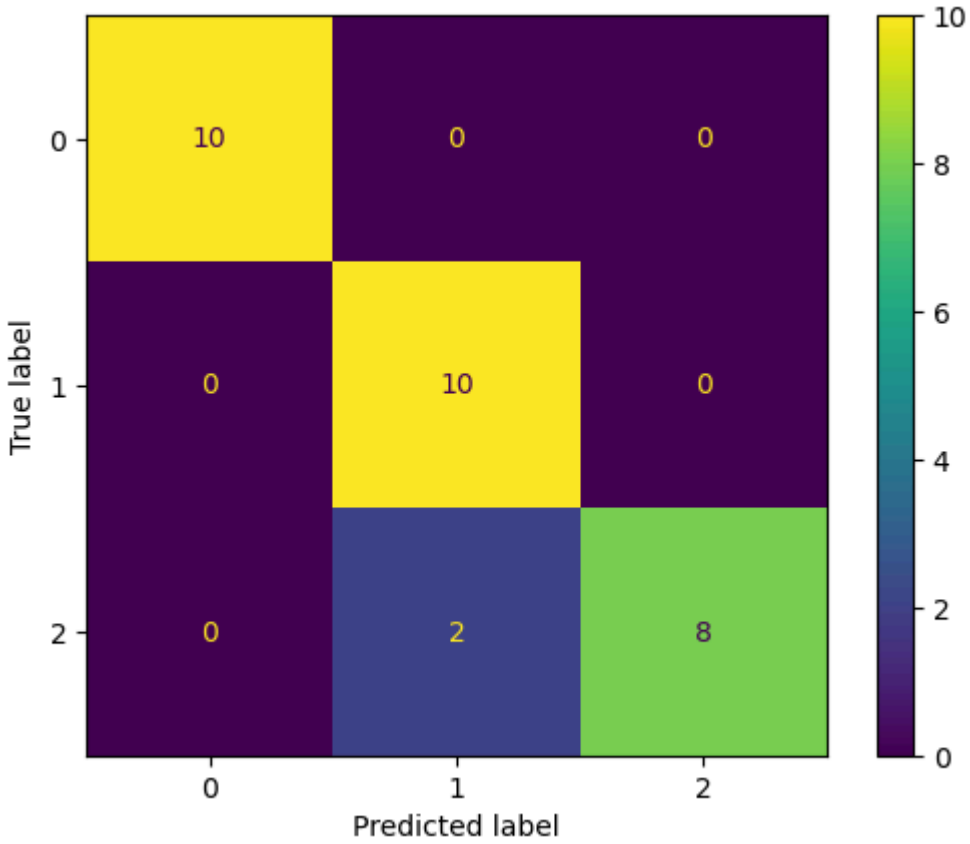
In [11]:
```
cl=0 #class we choose

t_pos=np.size(np.where((y_pred==cl) & (y_test==cl)))
f_pos=np.size(np.where((y_pred==cl) & (y_test!=cl)))
f_neg=np.size(np.where((y_pred!=cl) & (y_test==cl)))

precision=t_pos/(t_pos+f_pos)
recall=t_pos/(t_pos+f_neg)
F1=2*precision*recall/(precision+recall)

print(f"Metrics calculated manually:\n\tPrecision(c) = {precision},\n\tRecall(c) = {recall},\n\tF1(c) = {F1}.")
```

```
Metrics calculated manually:
        Precision(c) = 1.0,
        Recall(c) = 1.0,
        F1(c) = 1.0.
```

In [16]:
```
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
print(f"Classification report:\n{classification_report(y_test, y_pred)}")
cm=confusion_matrix(y_test, y_pred)
print(f"Confusion matrix:\n{cm}")
print("\nIt's visualisation:")
ConfusionMatrixDisplay(confusion_matrix=cm).plot()
```

```
Classification report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       0.83      1.00      0.91        10
           2       1.00      0.80      0.89        10

    accuracy                           0.93        30
   macro avg       0.94      0.93      0.93        30
weighted avg       0.94      0.93      0.93        30

Confusion matrix:
[[10  0  0]
 [ 0 10  0]
 [ 0  2  8]]

It's visualisation:
```

Out[16]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x76859449ff90>




## Task 8. (Different values of $k$ and distance metrics)

Now, let us test the performance of kNN for different values of the parameter $k$ and distance metrics.

---

**Instructions:**

- Loop over $k \in 2, 3, \ldots, 10$ and distance metrics ($L^1$ = Manhattan, $L^2$ = Euclidean).
- For each combination of $k$ and metric, train a kNN model and compute the F1-score (use `f1_score` from `sklearn.metrics`).
- Store the results in a DataFrame with columns:
  - `metric` (1 or 2),
  - `k` (number of neighbors),

    - `F1` (F1-score).
- Identify the combination of $k$ and metric that gives the best F1-score.

---

**Remark:**

In practice, instead of writing such loops manually, we can use tools like `GridSearchCV` from `sklearn.model_selection` to search over many hyperparameters automatically (for example, different values of $k$, distance metrics, or weighting schemes).

In [19]:
```
from sklearn.metrics import f1_score

k_values=np.arange(9)+2
metrics=["manhattan", "euclidean"]
results=[]
for k in k_values:
    for m in metrics:
        km_knn=KNeighborsClassifier(n_neighbors=k, metric=m)
        km_knn.fit(X_train_scaled, y_train)
        y_km_pred=km_knn.predict(X_test_scaled)
        f1=f1_score(y_test, y_km_pred, average="weighted")
        results.append([k,m,f1])


df_results=pd.DataFrame(results, columns=["k", "metric", "F1"])
print(df_results)
best_combination=df_results.loc[df_results["F1"].idxmax()]
print(f"\nBest combination is for k={best_combination['k']} and {best_combination['metric']} metric, giving the F1 score of {round(best_com
```

```
     k     metric        F1
0    2  manhattan  0.932660
1    2  euclidean  0.932660
2    3  manhattan  0.966583
3    3  euclidean  0.932660
4    4  manhattan  0.897698
5    4  euclidean  0.932660
6    5  manhattan  0.932660
7    5  euclidean  0.932660
8    6  manhattan  0.932660
9    6  euclidean  0.932660
10   7  manhattan  0.966583
11   7  euclidean  0.966583
12   8  manhattan  0.966583
13   8  euclidean  0.932660
14   9  manhattan  0.933333
15   9  euclidean  0.966583
16  10  manhattan  0.899749
17  10  euclidean  0.966583

Best combination is for k=3 and manhattan metric, giving the F1 score of 0.966583.
```